PARALLEL DETECTION OF STRONGLY CONNECTED COMPONENTS IN MASSIVE GRAPHS: THE MULTISTEP METHOD

GEORGE M. SLOTA* AND SIVASANKARAN RAJAMANICKAM†

Abstract.
Finding the strongly connected components (SCCs) of a directed graph is a fundamental problem used in many fields. For example, it is part of computing the block triangular form of a sparse matrix. Tarjan's algorithm is an efficient serial algorithm, but relies on depth-first search which is hard to parallelize. Several parallel algorithms have been proposed, but show poor load balance and slow performance on many real world graphs. This paper introduces the Multistep Method, which combines several parallel SCC finding algorithms to deliver performance results with up to 60× speedup over the serial Tarjan's algorithm, while being capable of fully decomposing a 1.8 billion edge graph in under two seconds. As a breadth-first search is a key component for finding SCCs, we also demonstrate a fast implementation which delivers graph traversal speeds up to the equivalent of 18 billion edge traversals per second on a single 16 core shared-memory machine.

1. Introduction. The strongly connected components (SCCs) of a directed graph are the maximal subgraphs where every vertex within the subgraph can reach and can be reached by every other vertex within the subgraph. The decomposition of a directed graph into its strongly connected components is a useful analytic tool in many applications like social network analysis [17], compiler design, radiation transport solvers [16], and computing the block triangular form for linear solvers and preconditioners [19, 21].

The optimal sequential strongly connected component algorithm is based on linear time Tarjan's algorithm [20]. Tarjan's algorithm depends on a depth first search of the graph and is not an easy algorithm to parallelize with good scalability. There are task-based algorithms to compute the SCC [22, 16] for distributed memory computers and coloring based algorithms for both distributed memory and accelerator based approaches [18, 3].

The increase in concurrency available within a single compute-node allows us to explore shared-memory based algorithms for different problems. In addition, the increase in memory footprint allows us to analyze very large graphs that would have been impossible a few years ago using shared-memory-based algorithms. With a difficult-to-parallelize algorithm in distributed memory, a task-based algorithm that can exploit existing task-based shared memory programming models, and a coloring algorithm which depends on fine-grained atomics, the problem of computing the strongly connected components is one of the ideal problem to stress test shared-memory implementations.

Our contributions in this paper include an optimized implementation of existing parallel algorithms using shared-memory programming models, a new parallel algorithm for computing the SCCs for real world graphs that combines the benefits of previous algorithms while at the same time reducing and is up to ∼11× faster on average with 16 cores, a fast atomic and lock-free breadth-first search (BFS) implementation that can reach the equivalent of over 18 billion edge traversals per second (GTEPS) in a single shared-memory node, and an experimental evaluation of several different parallel algorithms.

The rest of the paper is organized as follows. Section 2 describes the existing serial and parallel algorithms for computing SCCs. Section 3 describes our multistep method

*The Pennsylvania State University, gms5016@psu.edu
†Sandia National Laboratories, srajaman@sandia.gov

to compute SCCs. Sections 4 cover the implementation details in shared-memory architectures and Sections 5 and 6 demonstrate the benefits our new algorithms for various real-world graphs.

2. Background. There are several existing serial and parallel algorithms that have been used to determine the SCCs of a graph. This section will give a general overview of several of the most widely used ones, including Tarjan's and Kosaraju's algorithms, the Forward-Backward algorithm, Coloring, and Trimming.

2.1. Serial Algorithms. The primary serial algorithms used for determining strongly connected components within a graph are Tarjan's [20] and Kosaraju's [1] algorithms. Tarjan's algorithm uses a recursive depth first search (DFS) to form a search tree of explored vertices. The roots of the subtrees of the search tree form roots of strongly connected components. Kosaraju's algorithm performs two passes of the graph. It initially performs a DFS, placing each vertex onto a stack after it has been fully explored. After all vertices have been placed onto the stack, a vertex is popped from the stack and a DFS or BFS search is performed on the transpose of the graph. All vertices that can be reached by this vertex (that have not already been explored a second time) form an SCC.

Although the initial DFS in Kosaraju's algorithm is very difficult to parallelize, the second searches of the transpose can be parallelized quite trivially, due to the fact that the second search needs only to determine reachability, rather than any specific ordering. As Tarjan's algorithm only performs a single search of the entire graph, its work is $O(n+m)$, where n is the number of vertices and m is the number of edges in the graph, and can therefore be considered asymptotically optimal.

2.2. Parallel Algorithms. As the above described serially algorithms are non-trivial to parallelize in a scalable way, different algorithms have been proposed that allow parallel computation of strongly connected components. Two of the most widely used are the recursive Divide-and-Conquer Strong Components algorithm [22] (which has become more widely known as the Forward-Backward algorithm) and the iterative coloring algorithm [18]. We will also discuss the trimming procedure which can be used to quickly eliminate trivial SCCs and decrease the overall amount of work necessary for the primary algorithm. The trimming procedure can be used as a pre-processing step for any of the algorithms described here.

2.2.1. Forward-Backward Algorithm. The Forward-Backward (FW-BW) algorithm is given in Algorithm 1 and can be described as follows: Given a graph G, a single *pivot* vertex v is selected. This can be done either randomly or through simple heuristics. A BFS/DFS search is conducted starting from this vertex to determine all vertices which are reachable (forward sweep). These vertices form the *descendants* set (D). Another BFS/DFS search is performed from v but on the transpose graph. This search (backward sweep) will find the set (P) of all vertices than can reach v called the *predecessors*.

The intersection of these two sets form an SCC ($S = D \cap P$) that has the pivot v in it. If we remove all vertices in S from the graph, we can have up to three remaining distinct sets: $(D \setminus S), (P \setminus S)$, and *remainder* R, which is the set of vertices that we have not explored during either search from v. The FW-BW algorithm can then be recursively called on each of these three sets.

Possible parallelism exists for this algorithm on two levels. Primarily, as each of the three sets are distinct, they can be further explored in parallel. In addition, each

Algorithm 1 Forward-Backward Algorithm

```
1: procedure FWBW(G)
2:     if G = ∅ then
3:         return ∅
4:     end if
5:     select pivot v
6:     D ← DESC(G, v)
7:     P ← PRED(G, v)
8:     R ← (G \ (P ∪ D)
9:     S ← (D ∩ P)
10:    S ← S ∪ FWBW((D \ S))
11:    S ← S ∪ FWBW((P \ S))
12:    S ← S ∪ FWBW(R)
13:    return S
14: end procedure
```

of the forward and backward searches can be trivially parallelized using a standard parallel BFS or even DFS (since we don't care about ordering, only reachability).

2.2.2. Coloring. The coloring algorithm is given in Algorithm 2. This algorithm is similar to FW-BW in that it proceeds in forward and backwards passes. However, the approach is quite different, as it generates multiple pivots, or more commonly termed, *roots*, during the forward phase and only looks at a subset of G for each pivot on the backwards phase.

Algorithm 2 Coloring Algorithm

```
1: procedure COLORING(G)
2:     while G ≠ ∅ do
3:         initialize colors(v_id) = v_id
4:         while not fully colored do
5:             for all v ∈ G do
6:                 for all n ∈ N(v) do
7:                     if colors(v) > colors(n) then
8:                         colors(n) ← colors(v)
9:                     end if
10:                end for
11:            end for
12:        end while
13:        for all unique c ∈ colors do
14:            SCC(c_v) ← PRED(G(c_v), c)
15:            G ← (G \ SCC(c_v))
16:        end for
17:    end while
18: end procedure
```

Given a graph G, the algorithm starts by assigning a set of unique (numeric) *colors* to all the vertices, most easily as the vertex identifiers v_{id} for all $v \in G$. These colors are then propagated outwardly from each vertex in the graph. If a vertex v has any neighbors $n \in N(v)$, where N is the neighbor list, with a color lower than it's

own, the neighbors' colors are updated to that of the vertex. This process continues until no more vertices change their color.

We have now effectively partitioned the graph into distinct sets with separate colors c. As we started with v_{id} as our colors, for each distinct c, there is a unique vertex c_v with that identifier. We consider c_v as the root of a new SCC, $SCC(c_v)$. The SCC is then all vertices that can be reached backward from c_v that are also colored with the same color. We remove $SCC(c_v)$ from G, find the rest of the SCCs for all c in the current iteration, and then proceed to the next iteration and continue until G is empty.

Parallelizing this algorithm is easy, as both the forward coloring step across all v and the backward SCC step across all c_v can be parallelized quite trivially.

2.2.3. Trimming Procedure. The trimming procedure was initially proposed as an extension to FW-BW [16], to remove all trivial strongly connected components. The procedure is quite simple as shown in Algorithm 3: all vertices that have an in degree or out degree of zero (without self-loops) are removed. This can be performed iteratively or recursively as well, as removing a vertex will change the effective degrees of it's neighbors. We call performing a single iteration of trimming as *simple* trimming and performing trimming iteratively as *complete* trimming. This procedure is very effective in improving the performance of the FW-BW algorithm, but can be beneficial to use before running other algorithms, as well.

Algorithm 3 Trimming Algorithm

```
1:  procedure TRIM(G)
2:      while not fully trimmed do
3:          for all v ∈ G, where valid(v) = true do
4:              if degree_in(v) = 0 then
5:                  valid(v) ← false
6:              else if degree_out(v) = 0 then
7:                  valid(v) ← false
8:              end if
9:          end for
10:     end while
11: end procedure
```

2.3. Other Work. There has been other more recent work aimed at further improving upon the preceding algorithms, as well as developing newer algorithms to further decompose the graph and/or improve performance and scalability.

One example is the OBF algorithm of Barnat et al. [4], which, like coloring, aims at every iteration to further decompose the graph into multiple distinct components each containing a single SCCs. The OBF decomposition step can be performed much quicker than coloring, however. Barnat et al. [3] further implemented the OBF algorithm as well as FW-BW and coloring on the Nvidia CUDA platform, demonstrating considerable speedup over equivalent CPU implementations.

More recently, Hong et al. [11] demonstrated several improvements to the FW-BW algorithm and trimming procedure through expanding trimming to find both 1-vertex and 2-vertex SCCs, further decomposing the graph after the first SCC is found by partitioning based on connected components, and implementing a dual-level task-based queue for the recursive step of FW-BW to improve times and reduce overhead

for task-based parallelism.

3. The Multistep Method. We will now introduce our algorithm for graph SCC decomposition, the Multistep method. The reason for this name comes from the fact that it is a combination of the previously described algorithms stepped through in a certain order. This section will give our justifications for developing the algorithm in this way, as well as provide detail into our algorithm's specifics.

3.1. Observations. The FW-BW algorithm can be quite efficient if a graph has relatively small number of large and equally-sized SCCs, as the leftover partitions in each step would on average result in similar amounts of parallel work. The FW and BW searches could also be efficiently parallelized in this instance.

However, the structure of most real world graphs is very different. From observations, most real-world graphs have one giant SCC containing a large fraction of the total vertices, and many many small SCCs and often disconnected SCCs that remain once the large SCC is removed [17]. Running a naïve implementation of FW-BW would result in a large work imbalance after the initial SCC is removed, where the partitioning of the remaining sets would be heavily dominated by the remainder set. Additionally, using a naïve task-based parallelism model would add considerable overhead as each new task might only be finding an SCC of a few vertices in size before completing. In general, the size of the recursive tree and therefore the overall runtime of the FW-BW algorithm is dominated by the total number of SCCs that are in the initial graph.

Conversely, the coloring algorithm is quite efficient when the graph contains a large number of small and disconnected components, as the runtime of each step is dominated by the diameter of the largest connected component in the graph, or the number of steps needed to do the full coloring. This also leads to very poor performance on real-world graphs, as the time for each iteration can be very large while the largest SCCs remain, and there is no guarantee that these SCCs will be removed in any of the first few iterations.

It is also important to note that below a certain threshold of the number of vertices in a graph, the general parallel overhead results in worse performance when compared to simply using Tarjan's or Kosaraju's serial algorithm.

3.2. Description of Method. Based on the above observations, we therefore have developed what we term as the Multistep Method. This method aims at maximizing the advantages and minimizing the drawbacks of trimming, FW-BW, coloring, and a serial algorithm by applying them in sequence to decompose large real-world graphs into their strongly-connected components.

An overview of the algorithm is given in Algorithm 4. There are four primary phases of the algorithm. The first phase is trimming. We choose to do only a single iteration of trimming, as experiments have shown that the vertices trimmed in the second or subsequent iterations can be more efficiently handled in the coloring or serial phases.

In the second phase, we select an initial pivot vertex as the vertex in the graph that has the largest multiplicative sum of its in degree and out degree. This is an attempt to increase the chances that our initial pivot is contained within the largest SCC. Although there is no guarantee to ensure that, in practice with real world graphs it acts as a very good heuristic.

With the chosen pivot we do one iteration of modified FW-BW algorithm. Our changes to the FW-BW iteration avoid considerable work during this phase by not

Algorithm 4 Multistep Algorithm

```
1: procedure MULTISTEP(G)
2:     Trim_simple(G)
3:     select pivot v = max(v_dg), where v_dg = d_in(v) * d_out(v)
4:     FWBW_SCC(G, v)
5:     while NumVerts(G) > V_cutoff do
6:         ColoringStep(G)
7:     end while
8:     Tarjan(G)
9: end procedure
```

computing the three sets D, P and R and leaving all the vertices that are not part of the one SCC we computed to the next step.

As we do not partition the vertices into three sets, but compute just one SCC, we do a full forward search from the pivot and on the backward search explore vertices that were already marked as explored on the forward search. While doing a BFS for the backward search, if we encounter a vertex not already marked, we avoid adding it to the next level queue.

For a simple proof as to why this will work, assume by contradiction that a predecessor vertex p_i we find during the backward phase that was not marked during the forward phase has a predecessor p_j that was marked during the forward phase. This cannot happen as if the predecessor was previously marked, then the original predecessor p_i, being a descendant of p_j, would therefore have been marked as well. Since we know that, in order to be in the SCC, a vertex must be marked as both a descendant and predecessor, then we can safely ignore these p_i. For certain graphs, this can result in a considerably shorted search during the backward phase.

At the end of the FW-BW step, we simple take the remaining vertices and pass them all off to coloring. We run coloring until the number of vertices remaining crosses below a certain threshold determined experimentally, and then pass off the still remaining vertices to the final step, which is just Tarjan's serial algorithm.

4. Implementation Details and Optimizations. This section will provide a bit more detail into some of the implementation specifics. All code was written in C++ using OpenMP for shared-memory parallelization. We achieve most of our performance by using thread-owned queues, boolean arrays, and various techniques to avoid work, as well as avoiding atomic or locking operations whenever possible by simply accounting for any possible (and usually minor) race condition.

4.1. Graph Representation and other Data Structures. We use a standard compressed sparse row representation for our graph. Since most of our algorithms require both in and out edges, $O(2m + 2n)$ storage is required for both the in and out edge and degree lists. In order to avoid having to explicitly modify the graph structure, we keep a boolean array to signify whether a given vertex is still valid or not and unmark the vertex when the SCC it is contained in is completely found. We also maintain a current running list of the numeric identifiers of which vertices are still active, to greatly speed up loops that look across all vertices remaining in the graph.

In order to track which SCC a vertex in, another integer array is maintained. At the start of finding a new SCC (FW-BW, each root in each coloring step, each root

in Tarjan's) a running atomically-updated count value is incremented and noted. If a vertex is found to be in that certain SCC this noted value is assigned to it. All trimmed vertices are assigned the same value for simplicity.

4.2. Trim Step. We consider two different approaches for trimming. For simple trimming, we only need to look at the degree values as initially set when the graph was created. Therefore, we just need to do a single pass through all vertices, retrieve their in/out degrees, and flip their valid boolean if either is zero.

Complete trimming is a bit more complex. To greatly speed up processing during complete trimming, we create current and future queues and an additional boolean array of values to signify for each vertex if they are currently placed in the future queue. We place all vertices in the current queue to begin with.

We then determine the in and out degrees for all vertices contained in the current queue. If a vertex has an effective in or out degree of zero, then it is marked as no longer valid. Additionally, any valid vertices that the removed vertex was pointing to or had pointing at it is then placed in the future queue and marked as such. After the current queue is empty, the queues are swapped, the marks reset, and this process is repeated for as many iterations as necessary. The queues are used to avoid having to look through all vertices at each iteration, as it has been observed that the long *tendrils* [6] of vertices in lots of real world graphs tend to result in long tails of iterations where only a few vertices are removed at a time. The marking is done to prevent a vertex from being placed in the future queue multiple times. To avoid the synchronization overhead that would be required with a parallel queue, we maintain separate queues for each thread and combine them into the next level queue at the end of each step. We perform a similar operation with our BFS, as we will explain later.

As we will later show, that although complete trimming is easily paralellizable and can be quite fast, with the queues and marking being done very similarly to how we will soon describe our BFS, we find simple trimming to generally perform better overall. It is observed that a single iteration will remove the vast majority of vertices than *can* be removed, it does not require the explicit calculation or tracking of changing degrees at each iteration, and does not need queues or other such structures to maintain. Simply passing off the vertices not trimmed in the first iteration to be handled by coloring or the serial algorithm gave us better performance for all tested graphs.

4.3. Large SCC Search Step. For most graphs, determining the initial large SCC takes a majority of the overall runtime. Because this initial search is essentially just two breadth-first searches, we have developed and attempted to optimize for performance a BFS using techniques developed recently from a number of sources.

4.3.1. Parallel BFS. An overview of the BFS used in our Multistep approach is given in Algorithm 5. We will now describe some of the optimizations and design choices we implemented.

A typical BFS optimization is to use a bitmap of length n to signify whether or not a vertex has already been visited, and to avoid further exploring the vertex if it has been. A bit vector is able to fit completely in the last level cache on modern server-grade CPUs for graphs of up to tens of millions of vertices. It is assumed that by staying in cache, a quick boolean check is possible and accesses to main memory are minimized.

However, our experimentation has demonstrated that a boolean array actually

Algorithm 5 Multistep BFS

```
1: procedure MultistepBFS(G, p)
2:     queue ← p
3:     do_hybrid ← false
4:     while queue ≠ ∅ do
5:         if hybrid = false then
6:             for all v ∈ queue do in parallel
7:                 for n ∈ N_out(v) do
8:                     if visited(n) = false then
9:                         visited(n) ← true
10:                        thread_queue ← n
11:                    end if
12:                end for
13:            end for
14:        else
15:            for all v ∈ G do in parallel
16:                if visited(v) = false then
17:                    for all n ∈ N_in(v) do
18:                        if visited(n) = true then
19:                            visited(v) ← true
20:                            thread_queue ← v
21:                            break
22:                        end if
23:                    end for
24:                end if
25:            end for
26:        end if
27:        Synchronize
28:        Single thread does
29:            hybrid ← EvaluateHybridSwitch()
30:            if hybrid = false then
31:                for all thread_queue do
32:                    for all v ∈ thread_queue do
33:                        queue ← v
34:                    end for
35:                end for
36:            end if
37:        end single thread
38:        Synchronize
39:    end while
40: end procedure
```

outperforms a bitmap for our test environment. The likely reason for this is two-fold. Firstly, in order to calculate the address for a specific bit, at least an integer division, remainder operation, and bit shift is required. Additionally, since the CPU on our testbed only guarantees atomic read/writes starting only at the byte level [12, s. 8.1.1], either explicit atomic operations are needed or a more complex bit read/write function is required [9]. By using a byte-addressed boolean array and avoiding explicit locks, we see considerably faster runtimes. Although avoiding explicit locks might result in extra work (two threads see a vertex as unexplored, set the same bool to the same value, and put the vertex in the next level queue twice), experimentation has shown this to be of minimal concern.

As mentioned previously, we avoid additional locks and atomic operations by giving each threads its own next level queue. At the end of each level, the vertices from each thread queue are collected and placed in the frontier for the next level by a

single thread. Although this is necessarily done is serial, experimentation has shown it to be very fast, taking on the order of milliseconds even for millions of nodes, and greatly outperforming a shared queue with locks. This step can also be skipped when we are going to run the hybrid bottom-up BFS, which will be discussed next.

4.3.2. Hybrid BFS Approach. A hybrid bottom-up approach to the BFS was recently introduced by Beamer et al. [5]. They noted that at certain levels of a BFS in real world graphs (small world, scale free), it is actually more efficient to simply look in the reverse direction. Instead of all vertices currently on the frontier looking at all their children, all unvisited vertices simply attempt to find their parent on the frontier. It is not even necessary to check if the parent is explicitly on the frontier, but only if the parent has already been marked as visited (the child would have already been discovered if the parent's level is one previous to that of the current frontier).

Beamer et al. found that this approach will vastly decrease the total number of edge examinations needed during the BFS, improving search times by over $3\times$. Upon implementing their approach as described with parameters ($\alpha = 15, \beta = 25$), we noticed considerable speedup as well. A different design choice we had to implement was to maintain the thread queues while we are currently running the bottom-up hybrid as opposed to explicitly rebuilding the queue from scratch when we switch the hybrid off. This is due to the fact that we do not maintain any explicit BFS tree as we only require the visited array to determine the SCC, so we have no ability to track BFS level on a per-vertex basis.

4.3.3. Graph Partitioning Across Sockets. A final optimization we implemented was a per-socket graph partitioning and exploration scheme similar to the ones described in Agarwal et al. [2] and Chhugani et al. [9]. In this, we attempt to reduce a majority of cross-socket data travel by partitioning the graph across the memory closest to separate sockets, and only loading edges of a vertex for a thread running on the CPU closest to the socket by maintaining separate per-socket queues. We use the hwloc library [7] to enforce locality. At the end of each level, vertices discovered by a socket not containing its edges are passed over to the queue for the socket that has them. When we hit a level where we decide to switch to the bottom-up approach, the vertices that are examined by each socket are simply the ones that is contained in that partition.

Another approach we attempted for our two-socket Compton testbed was by placing all in degree data on one socket and all out degree data on the other, and then running both the forward and backward steps concurrently. We then attempted to include the bottom-up approach in this scheme, by enforcing level synchronization between the two searches and moving the boolean visited arrays across socket when the switch occurred. A variety of switch heuristics were explored, without much effect.

Although these partitioning approaches improved parallel scaling, it was only in a limited number of instances that actual runtimes improved due to the additional overhead. A wide variety of parameters and optimizations were explored, including reverting to actual bitmaps, partitioning based on number of vertices and number of edges, and even randomly permuting the input graphs to improve work balance, among others. However, no considerable and definitive improvements were ever noted over simply using our BFS with the bottom-up hybrid, so this approach was abandoned.

4.4. Coloring Step. The coloring step of our coloring algorithm is implemented quite similarly to our BFS and complete trimming algorithm, and is given in Algo-

rithm 6. Initially, all still valid vertices are assigned a color as their vertex id v_{id}, or index, in our graph structure. All valid vertices are then placed into the queue. For all vertices in parallel that are contained in the queue, we check to see if they have a higher color than their children. If they do, the color is passed to the child, and both the parent and child are placed in the thread's next level queue, and globally marked as such for all threads to see.

Algorithm 6 Multistep Coloring

```
1: procedure MultistepColoring(G)
2:     while NumVerts(G) > cutoff do
3:         for all v ∈ G do
4:             color(v) ← v_id
5:             queue ← v
6:             in_next_queue(v) ← false
7:         end for
8:         while queue ≠ ∅ do
9:             for all v ∈ queue do in parallel
10:                for n ∈ N_out(v) do
11:                    if color(v) > color(n) then
12:                        color(n) ← color(v)
13:                        if in_next_queue(n) = false then
14:                            in_next_queue(n) = true
15:                            thread_queue ← n
16:                        end if
17:                    end if
18:                end for
19:                if any n changed color then
20:                    if in_next_queue(v) = false then
21:                        in_next_queue(v) = true
22:                        thread_queue ← v
23:                    end if
24:                end if
25:            end for
26:            Synchronize
27:            Single thread does
28:                for all thread_queue do
29:                    for all v ∈ thread_queue do
30:                        queue ← v
31:                        in_next_queue(v) ← false
32:                    end for
33:                end for
34:            end single thread
35:            Synchronize
36:        end while
37:        for allunique c ∈ colors do in parallel
38:            SCC(c_v) ← PRED(G(c_v), c)
39:            G ← (G \ SCC(c_v))
40:        end for
41:    end while
42: end procedure
```

Why do we place the parent in the queue as well? To once again avoid explicit locks. It is possible and very likely that two parents will have higher colors than a shared child, creating a race condition. Both parents will once again examine their children to make sure that either the color that was given or a higher one has been placed. Additionally, since only a higher color can be assigned, we can ignore the race

condition created if a parent has their own color overwritten before they assign their previous one to the child. This approach has been experimentally determined to be much faster than explicit locks.

It was also tried to avoid locks instead by going bottom-up and having children look at their parents' and own color and take the largest, avoiding the race condition entirely. However, this is much slower in practice, because either all vertices need to be examined at each iteration, or the out vertices of the child need to be examined to create the queue, effectively doubling the amount of memory transfer needed at each iteration.

Our parallel SCC finding step is fairly standard, as it is a trivial algorithm to parallelize. We simply determine the root vertices by finding all unique colors in the graph, and then run a serial DFS on the transverse graph from each root, only looking at vertices with the same color as the root. We use a DFS here, since there isn't further room for parallelism, and experimentation has shown our serial DFS to be faster than our serial BFS.

4.5. Serial Step. We implement a simple and efficient recursive Tarjan's for our serial algorithm. We chose Tarjan's as our serial algorithm over Kosaraju's, based mainly on superior experimental runtimes. Although Kosaraju's can benefit from parallelization during the backwards step, the benefit when the graph has a small number of (usually) disconnected SCCs, is quite small if not negative.

We experimentally determined that a cutoff of about 100,000 remaining vertices is a relatively good heuristic for switching to the serial algorithm, although this is hardware specific. Some graphs benefit from running coloring all the way to completion, while some others would benefit more from switching to serial sooner. However, determining this cutoff without prior knowledge of the graph is quite difficult (possibly some calculation based on the number of steps needed to fully color the graph?), and the difference is usually close to negligible.

4.6. FW-BW Implementation. For comparison, we have also implemented the fully recursive FW-BW algorithm. For the recursive calls at the end of each FWBW procedure, we use task parallelism via `#pragma omp task`. Each recursive call recieves an array listing the numeric identifiers of vertices currently active in that call. As opposed to a boolean array identifying which vertices are globally still active, we use the integer array that was previously described as being used for marking which SCC each vertex is in. For each new call, all of the vertices active for that call have the same unique (for that specific call) value marked, which is retrieved from a global value. To check if a vertex is active for the call, we simply need to check what value it has marked. For the active vertices not found in the SCC for the current call, the global value is noted and atomically incremented by three, with the three skipped values being assigned to vertices based on if they are in the R, $(D \setminus S)$, or $(P \setminus S)$ set.

5. Experimental Setup. Experiments were performed on a 2 socket machine with 64GB RAM and Intel(R) Xeon(R) E5-2670 CPUs at 2.60GHz each having 20MB last level cache (*Compton*). Compton was running RHEL 6.1 and compilation was done with `icc` version 13.1.2. The `-O3` optimization parameter was used with the `-fopenmp` flag. Environmental variable `KMP_AFFINITY` and/or hwloc was used to control thread locality when needed.

Several large real world graphs were used in the course of this work. They are listed in Table 5.1. These graphs were retrieved from a number of sources, namely the SNAP Database [14], the Koblenz Network Collection [13], and the University

Network	n	m	d_{avg}	d_{max}	Dia.	# SCCs	max SCC
Friendster	66M	1.8B	53	5.2K	32	3M	63M
Italy Web	41M	1.2B	28	10K	~830	30M	6.8M
WikiLinks	26M	0.6B	23	39K	~170	6.6M	19M
HV15R	2M	183M	140	484	~85	25K	2M
LiveJournal	4.8M	69M	14	20K	18	970K	3.8M
WikiTalk	2.4M	5.0M	2.1	100K	9	2.2M	500K
CA Road Net	2.0M	2.8M	1.4	7	849	1.7M	302
Google Web	876K	5.1M	5.8	460	22	410K	430K
XyceTest	1.9M	8.3M	4.2	246	~91	400K	1.5M
R-MAT_20	560K	8.4M	15	24K	9	210K	360K
R-MAT_22	2.1M	34M	16	60K	9	790K	1.3M
R-MAT_24	7.7M	130M	17	150K	9	3.0M	4.7M
R-MAT 2v128e	2.0M	128M	121	8.7K	6	1.0M	1.0M
$G(n,p)$ 2v128e	2.0M	128M	64	107	6	1	2.0M

TABLE 5.1
Network sizes and parameters for all networks used in our analysis.

of Florida Sparse Matrix Collection [10]. The R-MAT [8] and $G(n,p)$ networks were generated with the GTGraph [15] suite using the default parameters.

Friendster and LiveJournal are social networks. Italy Web and Google Web are web crawls of the It domain and Google, respectively. WikiLinks is the cross-link network between articles on Wikipedia, and WikiTalk is a network of edits across user pages. XyceTest is a Sandia National Labs electrical simulation network and HV15R is a computational fluid dynamics matrix of a 3D engine fan. R-MAT_20/22/24 are R-MAT graphs of scale 20, 22, and 24, respectively, while R-MAT 2v128 is an R-MAT graph set with 2M vertices and 128M edges. $G(n,p)$ 2v128e is a $G(n,p)$ random graph of the same size. Finally, CA Road Net is a graph of the road network of California. For the graphs that were undirected (Friendster and CA road network) we assign a random direction to each edge based on a random boolean initially seeded with `srand(0)`. This was done to transform the problems from finding only connected components to that of finding strongly connected components.

These graphs were selected to represent a wide mix of graph size and structure. The number of SCCs and max SCC both play a large role in the general performance of SCC decomposition algorithms, while the average degree and graph diameter can have a large effect on the BFS necessarily used for these algorithms.

6. Results. In this section, we are going to compare our Multistep algorithm to the parallel baseline FW-BW and Coloring algorithms, as well as to serial Tarjan's algorithm. Additionally, we are going to give justification for the algorithmic choices we have made and their influences on performance.

6.1. Comparison of Algorithms. Figure 6.1 gives a comparison on runtime with 16 cores on Compton for Coloring, Forward-Backward, Multistep, and Tarjan's algorithm on several graphs. Multistep is run as previously described, doing simple trimming, the large SCC removal, coloring until 100K vertices remain, and then serial Tarjan's. Coloring and FW-BW both run complete trimming, as Coloring greatly benefits from it in the first steps, while FW-BW benefits from it in the later steps. As is apparent from the Figure, Multistep demonstrates the fastest performance on all

tested graphs with the exception of the CA road network, where the serial algorithm performs fastest.

The likely explanation for Multistep's performance on CA road network graph is twofold: Firstly, the network is not very large, so parallel overhead can dominate the runtime, contributing to Tarjan's superior relative performance. Secondly, the structure of the graph created by randomly assigning edges to the originally undirected network is very unique. Due to the low average degree and very wide diameter, random edge assignments creates a very high diameter connected component with many many small strongly connected components. This structure lends to particularly fast performance of coloring with very slow performance of FW-BW, as can be seen. This is not seen on Friendster due to it's much higher degree, which results in it maintaining its social-network-like structure with one massive and several small strongly connected components. It should also be noted the CA road network is the only graph tested where our initial pivot selection heuristic fails to land within the massive SCC, obviously due to the fact that it does not have one.

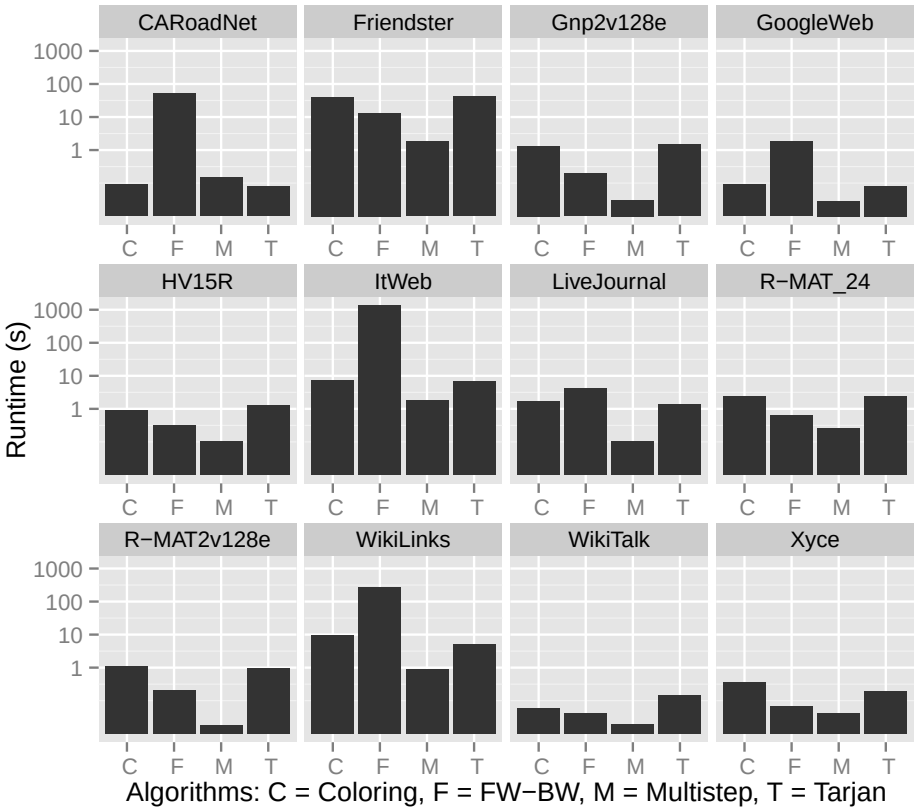
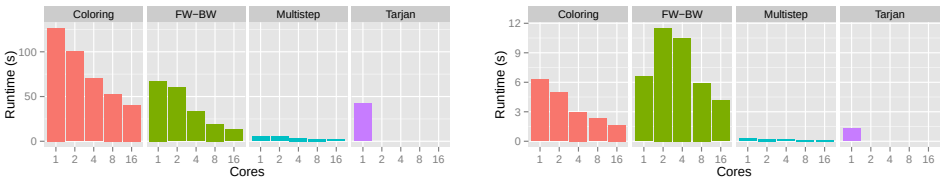


FIG. 6.1. *Runtimes on 16 cores for Coloring, FW-BW, Multistep, and Tarjan on a number of different graphs.*

As mentioned previously, the number of SCCs (after trimming), is the dominating factor for the runtime of FW-BW, while the largest diameter of the SCCs is the dominating factor for coloring. For graphs with very high numbers of small but non-trivial

SCCs (ItWeb, CARoad, WikiLinks), running FW-BW with naïve OpenMP task parallelism results in such high overhead that runtimes are several orders of magnitude greater than the other algorithms. Coloring however, shows fairly consistent performance, about running as fast as Tarjan's for all networks.

This effect is further illustrated in Figure 6.1, which shows parallel scaling of the algorithms on the Friendster and LiveJournal networks. Friendster only has several hundred SCCs after trimming, while LiveJournal has several hundred thousand. As can be seen on LiveJournal, switching from simple recursion in serial to two threads with task-based parallelism almost doubles the runtime. This is observed on several other graphs as well.



(a) Scaling on Friendster

(b) Scaling on LiveJournal

FIG. 6.2. *Parallel scaling of algorithms on Friendster (n=66M, m=1.8B, numSCCs=3M, maxSCC=63M) and LiveJournal (n=4.8M, m=69M, numSCCs=970K, maxSCC=3.8M)*

6.1.1. Strong Scaling. Figure 6.3 gives strong scaling of our Multistep approach on CA Road Net, HV15R, ItWeb, LiveJournal, R-MAT_24, R-MAT_2v128e, and Wikilinks. Running on 16 cores, we observe a 3× to 5× speedup on all graphs. Although we can utilize hyperthreading to run up to 32 threads on the test systems processors, we observe little to none (and sometimes worse) further speedup by doing so. Runtimes for 16 threads on 8 cores was observed to scale fairly well, and is within about 10% of the presented performance for all algorithms and graphs. As this is a heavily memory-constrained problem, and we are running on a dual socket system, we attempted a variety of different methods to improve scaling, which will be discussed later.

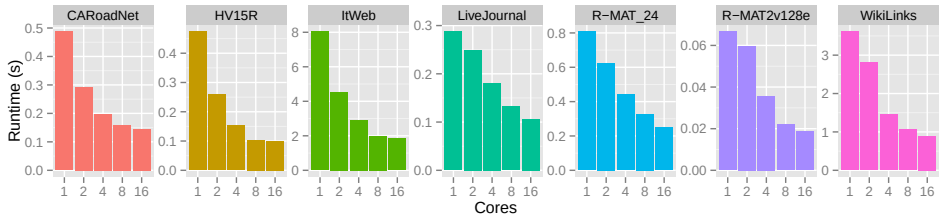


FIG. 6.3. *Parallel scaling of Multistep on up to 16 cores for several graphs.*

6.1.2. Weak Scaling. Finally, we analyze weak scaling of Multistep, Coloring, and FW-BW by using the R-MAT_20, 22, and 24 graphs. The number of vertices, edges, SCCs, and maximal SCC size increases by approximately 4× from R-MAT_20 to R-MAT_22 and from R-MAT_22 to R-MAT_24. Figure 6.4 gives the results. From this graph, it appears that our approach scales better than either Coloring or FW-BW.

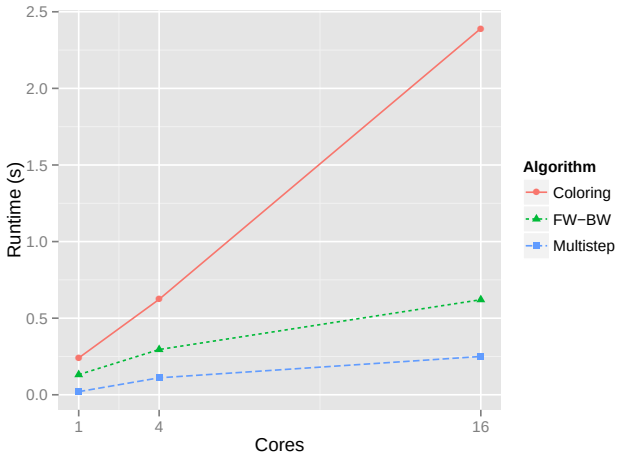


FIG. 6.4. *Approximate weak scaling across R-MAT_20/22/24 (n=560K/2.1M/7.7M, m=8.4M/34M/130M, numSCCs=210K/790K/3.0M, maxSCC=360K/1.3M/4.7M)*

6.2. Bottom-Up BFS and hwloc. In order to improve runtimes and scaling of Multistep beyond 16 threads, we attempted several different methods, the results of which are give in Figure 6.2. For these results, we consider the total time to do both the forward and backwards sweeps while finding the initial massive SCC.

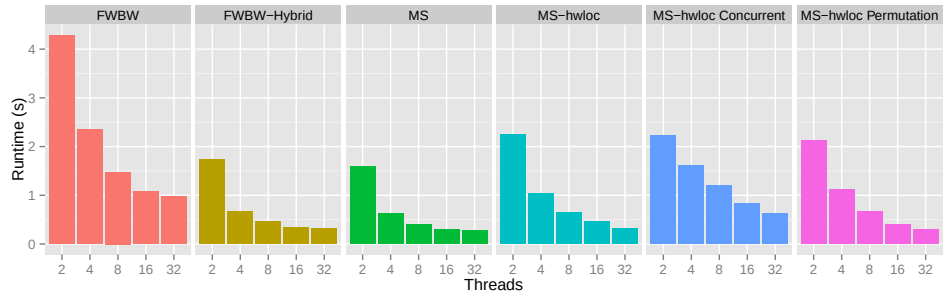
Our baseline approach (FWBW) is a simple parallel BFS utilizing our boolean 'bitmap'. In this, we do the complete forward and backward sweeps as would be done while running the FW-BW algorithm. The first improvement is the hybrid bottom-up approach as described in a preceding section. An immediate 3× to 4× improvement is readily apparent on both graphs. Next, we decrease the work needed on the backward sweep by recognizing which vertices weren't discovered on the forward sweep and not further exploring them (MS). This is the approach we use for Multistep.

As can be seen on both graphs while using the FWBW-Hybrid and Multistep approaches, minimal to no improvement is seen with hyperthreading. In an attempt to improve parallel scaling, we also investigate partitioning our graph across both sockets.

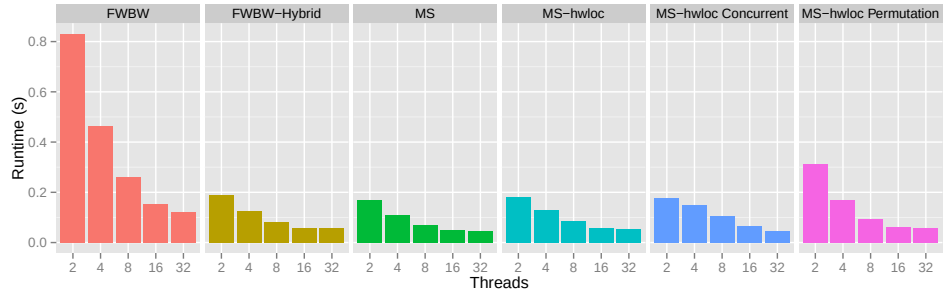
The first attempted method (MS-hwloc) simply partitioned the graph based on an even split of the edge adjacency lists. Splitting based on the number of vertices was also investigated, but found to be much slower in practice due to severe balance issues. We also produced graphs with an even split of both vertices and edges by doing random permutations of vertices (MS-hwloc Permutation). However, runtimes degraded severely on smaller graphs by doing this, likely due to the loss of graph structure-based locality of vertices and edges. On the larger networks such as WikiLinks, the graph structure's locality doesn't play as much a role due to the wider spread of vertex IDs, so most new edges will need loaded from memory anyways. Although not attempted, a smarter partitioning or permutation methodology (perhaps an even/odd split?), might be more beneficial.

The final attempted partitioning method simply put the out edge adjacency and degree lists on one socket and the in edge lists on the other with both forward and backward sweeps being performed concurrently (MS-hwloc Concurrent).

As can be seen in both Figures, none of the hwloc-based methods, while improv-



(a) BFS methods on Wikilinks



(b) BFS methods on LiveJournal

FIG. 6.5. *Parallel scaling of algorithms on WikiLinks (n=26M, m=0.6B, numSCCs=6.6M, maxSCC=19M) and LiveJournal (n=4.8M, m=69M, numSCCs=970K, maxSCC=3.8M)*

ing scaling through all 32 threads, decreased runtimes overall. This is most likely attributable to all of the extra overhead required, such as explicitly determining ownership of each vertex, maintaining separate queues, etc.

6.3. Performance of BFS. Overall, our BFS implementation gives very fast single node performance, with the equivalent of over 18 GTEPS on the R-MAT2v128e graph. We use the term 'equivalent', as while a traditional BFS will examine every edge in the connected component it is exploring, the hybrid approach does not. However, we still calculate our GTEPS values as $Runtime/E_c$, where E_c is the number of edges that a traditional BFS would have visited. Additionally, it should be explicitly noted that we do not maintain a BFS tree, as we only care to mark the vertices to find the SCC. The performance of our BFS implementation on every test graph is given in Figure 6.6.

The performance of a standard BFS, and especially the bottom-up BFS, has a high dependence on the average degree in the graph. This strong relationship is exemplified in Figure 6.7, which correlates the GTEPS performance of each of our test graphs as a function of the graphs' average degrees.

6.4. Trimming. Finally, we give justification to our *simple* trimming procedure, as opposed to the complete trimming procedures employed in previous work. Figure 6.4 gives the runtimes at each level of our Multistep procedure for LiveJournal and WikiLinks utilizing simple, complete, as well as no trimming. For both of these graphs, as well as most other ones observed during testing, complete trimming

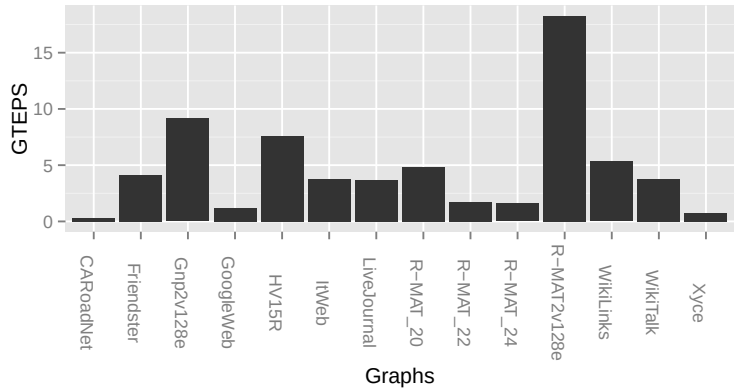


FIG. 6.6. *GTEPS performance of our BFS implementation on all test graphs.*

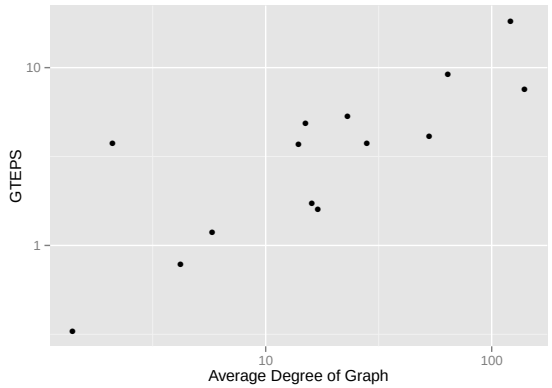
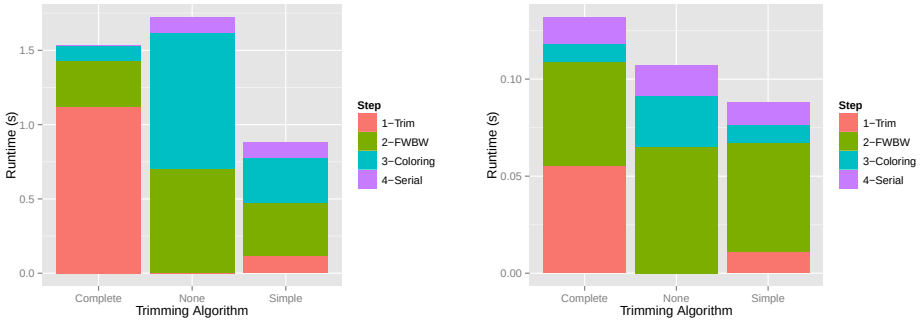


FIG. 6.7. *Correlation between BFS performance and the average graph degree.*

decreases the costs of the FW-BW and coloring steps. It has also been observed that complete trimming has the additional benefit of sometimes completely skipping the serial step, by letting coloring find all of the remaining SCCs in parallel in a single step when the graph still has over 100K active vertices remaining. Likely due to the fact that most trivial components are removed, letting only the larger and not usually inter-connected strong components, which coloring can easily find, remain.

However, the benefits of complete trimming never balance out the additional costs, in some instances resulting in overall performance worse than no trimming at all. The reason for this is due to the iterative nature of trimming, and the occurrence of the long *tails* of small numbers of vertices being trimmed during each iteration. This is especially apparent on certain graphs, such as WikiLinks and the It Web graph. A recursive approach, while faster in serial, is very difficult to parallelize. Additionally, complete trimming requires each examined vertex to look at all of its neighbors for validity to get the current effective in and out degrees, while simple trimming only needs to look once through both of the degree lists. A single iteration of simple trimming also removes a substantial majority of all possible vertices that can be removed for most of the tested graphs, which makes even running single extra



(a) Per-step times on WikiLinks

(b) Per-step times on LiveJournal

FIG. 6.8. *Justification for simple instead of complete trimming using WikiLinks and LiveJournal as examples*

iteration of complete trimming usually 'not worth it'.

7. Conclusions. This paper introduced the Multistep method, a combination of previous algorithms, for fast parallel strongly-connected component decomposition of billion-edged directed graphs. We also evaluated the existing algorithms which partially comprise the Multistep method, and offered several improvements and optimizations.

Future work might further investigate the possibility of enforcing hardware-based graph partitioning, especially on larger systems with multiple sockets and/or NUMA nodes. Additional improvements might also be made to our trimming and coloring procedures.

Acknowledgments. This work was supported by the DOE Office of Science through the FASTMath SciDAC Institute. Thanks are extended to Erik G. Boman for a number of helpful ideas, conversations, and support.

REFERENCES

[1] A. V. AHO, J. E. HOPCROFT, AND J. D. ULLMAN, *Data Structures and Algorithms*, Addison-Wesley, 1983.

[2] V. ARGARWAL, F. P. D. PASETTO, AND D. A. BADER, *Scalable graph exploration on multicore processors*, in Supercomputing, 2010.

[3] J. BARNAT, P. BAUCH, L. BRIM, AND M. CEVSKA, *Computing strongly connected components in parallel on cuda*, in Parallel and Distributed Processing Symposium (IPDPS), 2011 IEEE International, IEEE, 2011, pp. 544–555.

[4] J. BARNAT AND P. MORAVEC, *Parallel algorithms for finding sccs in implicitly given graphs*, Formal Methods: Applications and Technology, 4346 (2006), pp. 316–330.

[5] S. BEAMER, K. ASANOVIĆ, AND D. PATTERSON, *Direction-optimizing breadth-first search*, in Supercomputing, 2012.

[6] A. BRODER, R. KUMAR, F. MAGHOUL, P. RAGHAVAN, S. RAJAGOPALAN, R. STATA, A. TOMKINS, AND J. WIENER, *Graph structure in the web*, Computer networks, 33 (2000), pp. 309–320.

[7] F. BROQUEDIS, J. CLET-ORTEGA, S. MOREAUD, N. FURMENTO, B. GOGLIN, G. MERCIER, S. THIBAULT, AND R. NAMYST, *hwloc: a generic framework for managing hardware affinities in hpc applications*, in Proceedings of the 18th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP2010), 2010.

[8] D. CHAKRABARTI, Y. ZHAN, AND C. FALOUTSOS, *R-MAT: A recursive model for graph mining*, in SDM, 2004.

[9] J. CHHUGANI, N. SATISH, C. KIM, J. SEWALL, AND P. DUBEY, *Fast and efficient graph traversal algorithm for cpus: Maximizing single-node efficiency*, in Supercomputing, 2012.

[10] T. A. Davis and Y. Hu, *The university of florida sparse matrix collection*, ACM Transactions on Mathematical Software, 38 (2011), pp. 1–25.

[11] S. Hong, N. C. Rodia, and K. Olukotun, *Technical report: On fast parallel detection of strongly connected components (scc) in small-world graphs*, tech. rep., Stanford University, 2013.

[12] Intel, *Intel 64 and IA-32 Architectures Software Developer's Manual, System Programming Guide, Part 1*, vol. 3A, Intel Press, 2011.

[13] J. Kunegis, *KONECT - the koblenz network collection.* `konect.uni-koblenz.de`, last accessed 31 July 2013.

[14] J. Leskovec, *SNAP: Stanford network analysis project.* `http://snap.stanford.edu/index.html`, last accessed 3 July 2013.

[15] K. Madduri and D. A. Bader, *GTgraph: A suite of synthetic graph generators.* `http://www.cse.psu.edu/~madduri/software/GTgraph/`, last accessed 31 July 2013.

[16] W. Mclendon Iii, B. Hendrickson, S. J. Plimpton, and L. Rauchwerger, *Finding strongly connected components in distributed graphs*, Journal of Parallel and Distributed Computing, 65 (2005), pp. 901–910.

[17] A. Mislove, M. Marcon, K. P. Gummadi, P. Druschel, and B. Bhattacharjee, *Measurement and analysis of online social networks*, in Proceedings of the 7th ACM SIGCOMM conference on Internet measurement, ACM, 2007, pp. 29–42.

[18] S. Orzan, *On Distributed Verication and Veried Distribution*, PhD thesis, Free University of Amsterdam, 2004.

[19] A. Pothen and C.-J. Fan, *Computing the block triangular form of a sparse matrix*, ACM Transactions on Mathematical Software (TOMS), 16 (1990), pp. 303–324.

[20] R. E. Tarjan, *Depth first search and linear graph algorithms*, SIAM Journal of Computing, 1 (1972), pp. 146–160.

[21] H. K. Thornquist, E. R. Keiter, R. J. Hoekstra, D. M. Day, and E. G. Boman, *A parallel preconditioning strategy for efficient transistor-level circuit simulation*, in Computer-Aided Design-Digest of Technical Papers, 2009. ICCAD 2009. IEEE/ACM International Conference on, IEEE, 2009, pp. 410–417.

[22] W. McLendon III, B. Hendrickson, and S. J. Plimpton, *Finding strongly connected components in distributed graphs*, Lecture Notes in Computer Science, 1800 (2000), pp. 505–512.